高级语言程序设计

第3章：函数

计算机学院 陈建全
Email: cjq@nwpu.edu.cn

2024/2/8

- 3.1 黑盒
- 3.2 静态函数和静态变量
- 3.3 参数
- 3.4 返回值
- 3.5 工具箱、包、Javadoc和导入
- 3.6 函数的前置条件和后置条件*

- Java 中的函数可以是静态的或非静态的。本章介绍静态函数
- “黑盒”来形容代码模块或组件时，这通常意味着该部分的内部实现对于外部使用者来说是透明的或者隐藏的。用户或其他代码部分只需通过公开的API或接口与之交互，而不必担心其内部如何处理这些交互。这种设计思想与封装及抽象原则相一致，是高质量软件工程实践的标志。
- 黑盒的原则
  - 黑盒的接口应该简单、定义明确且容易理解。
  - 为了使用黑盒，你不必知道它如何实现的任何信息，只需要知道它的接口。
  - 黑盒的实现者也不需要了解使用黑盒的大系统的任何信息。

3.1 黑盒

- 函数就是一个黑盒，因为看不到它“里面”有什么，更准确地说，程序员通常也不想看到它里面的逻辑。
  - 把功能封装在函数中就是所谓的过程抽象
  - 借助于过程抽象实现的信息隐藏，使得函数内部可以被改变而不影响使用该函数的程序。信息隐藏也是黑盒原则的另一种说法。
- 一个类也是一个黑盒，一个类中可以有一个“公共”部分，代表它的接口，还有一个“私有”部分，它完全隐藏在实现中。
  - 把现实世界的概念通过类的形式进行模型化，并通过封装来隐藏内部细节，对外提供接口，是面向对象编程中数据抽象的体现。

3.2 静态函数和静态变量

- Java利用类把函数和变量封装在一起。
  - 不同类中的函数可以同名。
  - 类又按照包组织管理。
  - 这两个特点可以很好地解决大型项目的命名冲突问题。
- 类中的函数分为静态函数和非静态函数，它们之间存在较大的区别。一个类定义可以包含这两种类型的函数的源代码。
  - 静态函数又叫类方法，是类本身的成员.
  - 非静态函数又叫实例方法，仅在创建对象时使用，并且函数本身成为对象的成员。
  - 静态和非静态之间的区别也适用于类的成员变量。
  - 本章将主要讲述静态函数和静态成员变量，
- 类或对象中的函数通常也称为方法或者过程。

3.2.1 函数定义

◆函数定义的形式

```
<modifiers>
<return-type> <function-name> ( <parameter-list>)
{
    <statements>
}
```

```
[public | protected | private]
[static][ final][abstract] [native] [synchronized]
返回类型 方法名([参数列表]) [throws exceptionList]
{
        方法体
}
```

3.2.1 函数定义

◆函数定义的形式

- <statements>: 黑盒子的实现部分。函数被调用时其中的指令被执行。
- <modifiers>： 本章只考虑static和public
- <return-type>: 指定返回值的类型。如果没有返回值，写替换为void
- <parameter-list>: 函数接口的一部分，从外部传递给黑盒子的信息，供黑盒子内部计算用。

  参数列表可以是空的，也可以由一个或多个<type> <parameter-name>形式的参数声明组成, 用逗号“,”分隔。

  每个类型只能确定一个参数，比如两个int变量，只能写"int x, int y",不能写“int x,y"

◆静态方法只能直接访问同一个类中的静态变量和其他静态方法。静态方法不能直接访问类实例变量（非静态变量）或实例方法（非静态方法），因为这些实例成员要求在调用时存在一个具体的对象。

3.2.1 函数定义

◆例如，一个名为 Television 的类包含一个名为changeChannel()的方法，它的功能是改变频道，那么该函数运行后，可以使用一个参数来指定变更后的频道。如果频道号是整数，则参数类型为 int，changeChannel()的声明如下所示:

```
public void changeChannel(int channelNum) { ... }
```

◆函数参数本身是没有值的，调用时会提供值。比如:

```
changeChannel(17)
```

- 函数定义时的参数叫“形式参数” (formal parameters,简称parameters)。比如这里的channelNum。
- 函数调用时传递的值叫做“实际参数”(actual parameters, 简称argument)。比如这里的17。

3.2.1 函数定义

◆下面例子中，getnextN是一个非静态函数。

```
public static void playGame() {
    // "public"和"static" 是修饰符; "void"是返回值类型;
    // "playGame" 是函数名称;
    // ( )代表函数测参数是空，即函数没有参数.
    . . . // 函数 playGame 的语句.
}
int getNextN(int N) {
    // 函数没有修饰符; "int"是函数的返回值类型;
    // "getNextN" 是函数名称;
    //参数列表中只有一个参数，参数名为 N，类型是"int"
    . . . // 函数 getNextN 的函数体
}
static boolean moreThan(double x, double y) {
    // "static"是修饰符; "boolean"是返回值类型
    // "moreThan" 是函数名称;
    // 参数列表包含 2 个参数，参数名为"x"和 "y"，参数类型是"double"
    . . . // 函数 moreThan 的函数体
}
```

3.2.2 调用函数

◆对于静态函数，调用方法如下：

```
类名.静态方法名（参数）；//建议采用
对象名.静态方法名（参数）； //不建议这么用。
```

◆如果在同一个类内部调用，可以省略类名：

```
静态方法名（参数）
```

◆假设前面例子中 playGame()在类 Poker（扑克）中定义，从 Poker 类之外的地方调用它，其调用格式必须是：

```
Poker.playGame();
```

◆调用函数时的类名是告诉程序从哪个类中查找函数。

- 例如 playGame()可能在类 Mahjong(麻将)和 Chess（象棋）中都有定义，使用 Mahjong.playGame()或 chess.playGame()就可以区分调用哪个类的 playGame()函数。

◆静态方法调用举例:

```
public class E0301StaticMethodDemo {
    public static void main(String[] args) {

        System.out.println(Who());
        System.out.println(Parterner.Who());
    }
    public static String Who() {
        return "E0301StaticMethodDemo.Who()";
    }
}
class Parterner {
    public static String Who() {
        return "Parterner.Who()";
    }
}
```

```
$java E0301StaticMethodDemo.java
E0301StaticMethodDemo.Who()
Parterner.Who()
```

3.2.3 成员变量

- 函数中声明的变量被称为局部变量。
- 类中声明的变量不属于类中的任何函数，称类中的变量为成员变量。成员变量有时也叫做字段、属性。静态成员变量类似于C语言中全局变量。
- 变量可以是静态的也可以是非静态的
  - 静态成员变量又叫类变量。
  - 非静态成员变量又叫实例变量。

3.2.3 成员变量

◆静态成员变量可能被标记为公共或私有。例如：

```
static String usersName;
public static int numberOfPlayers;
private static double velocity, time;
```

◆未声明为私有的静态成员变量可以从定义它的类的外部以及内部访问。

- 例如，System类包含名为 out 的公共静态成员变量，可以通过引用 System.out 在自己的类中使用此变量。
- 同样，Math.PI 是 Math 类中的公共静态成员变量。

◆在函数中声明局部变量时，必须先为该变量赋值，然后才能对其进行任何操作。

◆成员变量会自动使用默认值初始化。

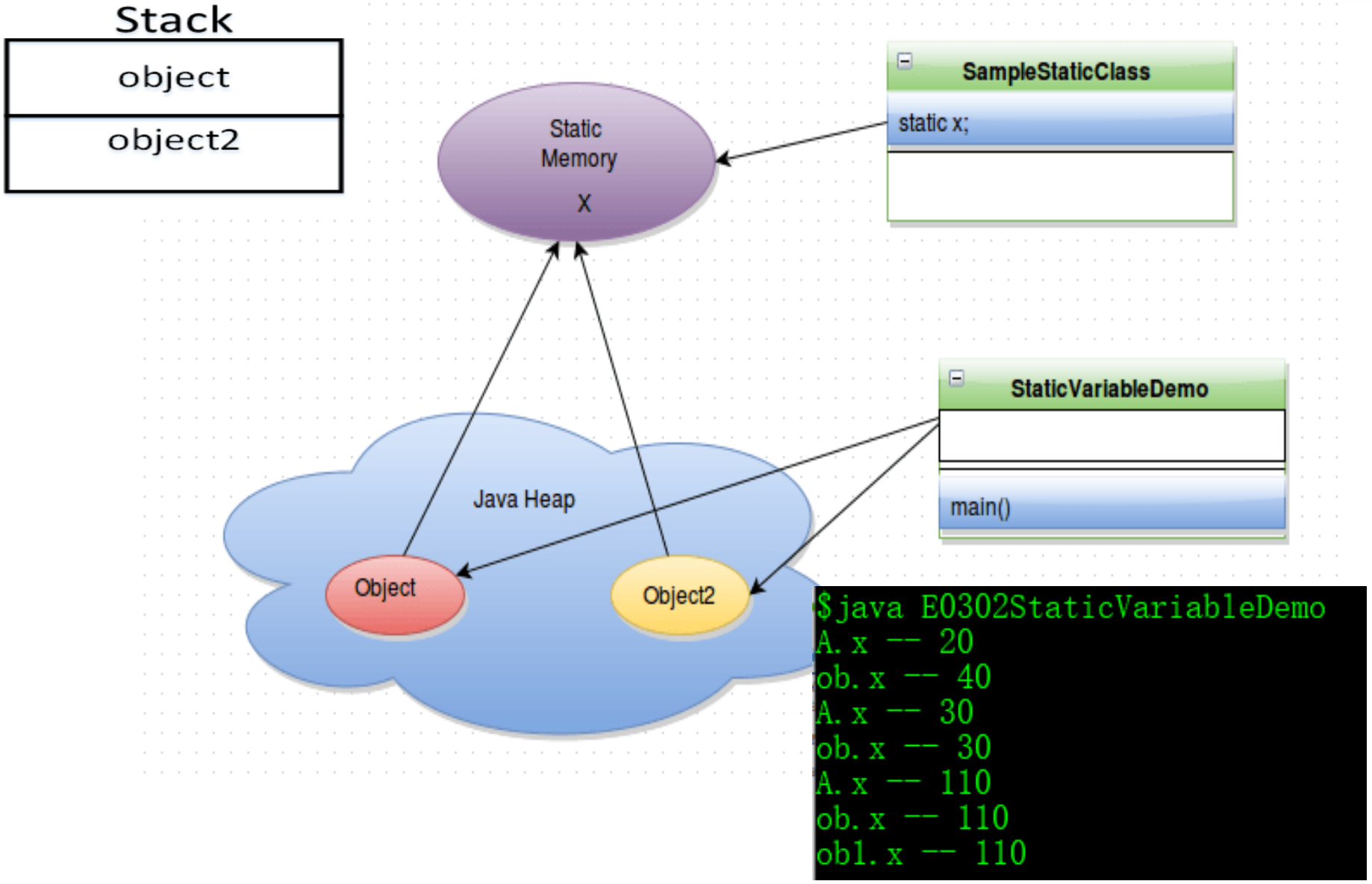
3.2.3 成员变量

- 静态成员变量作为一个整体属于类，只要类存在，它就存在。
- 当 Java 解释器首次加载类时，会为变量分配内存，任何为变量赋值的语句都会更改该内存的内容。
- 静态成员变量由类中的所有静态函数“共享”，即静态成员变量的值可以在一个函数中设置并在另一个函数中使用
- 下面的代码演示静态变量的共享：无论用类名引用还是用对象名引用，访问的都是同一个变量。

3.2.3 成员变量


```
class SampleStaticClass {static int x;  }
public class E0302StaticVariableDemo {
  public static void main(String[] args) {
    SampleStaticClass.x = 20; System.out.println("A.x -- " +
SampleStaticClass.x);
    SampleStaticClass object = new SampleStaticClass();
    object.x = SampleStaticClass.x + 20;
System.out.println("ob.x -- " + object.x);
    SampleStaticClass.x = object.x - 10;
System.out.println("A.x -- " + SampleStaticClass.x);
    System.out.println("ob.x -- " + object.x);
    SampleStaticClass object2 = new SampleStaticClass();
    object2.x = (object2.x * 2) + (SampleStaticClass.x - 10 +
object.x);
    System.out.println("A.x -- " + SampleStaticClass.x);
    System.out.println("ob.x -- " +
object.x);System.out.println("ob1.x -- " + object2.x);
}}
```

3.2.3 成员变量


Stack
object
object2
Static
Memory
X
SampleStaticClass
static x;
StaticVariableDemo
main()
Java Heap
Object
Object2
$java E0302StaticVariableDemo
A.x -- 20
ob.x -- 40
A.x -- 30
ob.x -- 30
A.x -- 110
ob.x -- 110
ob1.x -- 110

3.2.3 成员变量

◆这个例子中，scale为实例变量。这样在静态方法中访问scale时，发生如下编译错误（JDK11):

```
.\Converter.java:4: 错误: 无法从静态上下文中引用非静态 变量 scale
        return (cent * scale / 5 + 32);
                       ^
```

```
public class Converter {
    int scale=9;
    public static int centigradeToFahrenheit(int cent) {
    return (cent * scale / 5 + 32);
}
```

3.3.1 参数的使用

- 调用函数时，参数类型要”匹配“：类型一致，或者能够进行隐式类型转换。
- 例: 一个 3N+1 序列是根据以下规则计算的：“如果 N 为奇数，则将其乘以 3 并加 1；如果 N 为偶数，则将其除以 2；继续直到 N 等于 1为止。例如，当 N=3 时，得到序列：3, 10, 5, 16, 8, 4, 2, 1。
- 假设要编写一个函数来输出这样的序列，那么函数体将始终完成输出 3N+1 序列，但是函数输出的序列取决于 N 的值。因此，N 的值将是函数的参数。函数定义如下

3.3.1 参数的使用


```
  static void print3NSequence(int startingValue) {
    int N; // 序列值中的一个.
    int count; // 序列值的个数.
    N = startingValue; // 函数的输入参数作为第一个序列值
    count = 1; // 开始时，序列的个数为 1.
    System.out.println("The 3N+1 sequence starting from " + N);
    System.out.println();
    System.out.println(N); // 输出初始序列，即 N 的值
    while (N > 1) {
      if (N % 2 == 1) // is N odd?
          N = 3 * N + 1;
      else
          N = N / 2;
      count++; // 累加序列个数
      System.out.println(N); // 输出当前序列
    }
    System.out.println();
    System.out.println("There were " + count + " terms in the
sequence.");
  } // end print3NSequenc
```

3.3.1 参数的使用

◆说明：

■函数有一个参数” int startingValue “，表示序列的起点。

■函数体内部，参数和普通变量一样使用。

■但是函数内部并未给参数赋值，参数的值是函数被调用时，从外部传入的。

■比如如下调用：

```
print3NSequence(17);
```

这时会首先给startingValue赋值17,然后执行函数体。

■传递给函数参数的，除了字面量，还可以是表达式或者函数调用。比如

```
int k =17;
print3NSequence(k);  // startingValue为17
print3NSequence(k*2); //startingValue为34.
```

3.3.1 参数的使用

◆说明:

■比如如下一个main函数，以用户输入的整数为起点，输出序列。

■这个main函数需要和print3NSequence包装到同一个类中来使用。

```
  public static void main(String[] args) {
    System.out.println("This program will print out 3N+1
sequences");
    System.out.println("for starting values that you
specify.");
    System.out.println();
    int K; // 用户输入值 K，当 K < 0 时，循环停止.
    Scanner input=new Scanner(System.in);
    do {
      System.out.println("Enter a starting value.");
      System.out.print("To end the program, enter 0: ");
      K = input.nextInt(); // 得到初始值.
      if (K > 0) // 如果 K > 0，才输出序列.
          print3NSequence(K);
    } while (K > 0); // 如果 K > 0，进行下一次循环.
    input.close();
  } // end main
```

3.3.2 形式参数和实际参数

◆形参和实参

- 函数定义中的参数称为形式参数（formal parameters)或哑参数（ dummy parameters）。
- 调用函数时传递给函数的参数称为实参(actual parameters or arguments)。
- 当一个函数被调用时，函数调用语句中的实际参数被计算并且这些值被分配给函数定义中的形参，然后开始执行函数体。

◆形参类似一个变量，要有名字和类型，比如(int x, double d)

◆实参是一个值，可以由任何表达式指定，只要该表达式结果可以合法地赋值给形参。

◆比如如下函数：

```
static void doTask(int N, double x, boolean test) {
    ...// doTask 函数包含的函数体
}
```

3.3.2 形式参数和实际参数

◆函数调用语句如下：

■doTask(17, Math.sqrt(z+1), z >= 10);

◆相当于执行如下代码：

```
{
    int N; // 为形参分配内存地址.
    double x;
    boolean test;
    N = 17; // 第一个参数 N 的值赋值为 17
    x = Math.sqrt(z+1); // 计算 Math.sqrt(z+1)的值并赋值给变量 x
    test = (z >= 10); // 判断"z >= 10"的 true/false 并赋值给 test 变量
    doTask(17, x, test); // 函数 doTask 的调用语句
}
```

◆初学者容易犯的错误：在函数开头给形参赋值，或者要求用户输入形参的值。

■形参的值应该是调用时传递的实参,进入函数时已经有值。

3.3.3 全局和局部变量

- 函数中声明的局部变量、形式参数和在函数之外声明的静态成员变量
- 局部变量与外界无关，是函数内部的一部分。
  - 函数执行时，在栈中为局部变量分配空间。函数执行结束，局部变量占用的空间被释放。
- 形式参数的作用就很像局部变量。
  - 函数执行时，在栈中为参数分配空间。函数执行结束，参数占用的空间被释放。
  - 如果形式参数的类型是基本类型，那么在函数中对形式参数进行的更改对程序的其余部分没有影响；
  - 如果形式参数是数组或者其它对象，对形式参数内容的更改就会影响其它部分。（类似于C语言的指针参数）
- 与局部变量相反，静态成员变量可以在定义它的整个类中使用，如果它不是私有的，也可以在其他类中使用。对静态成员所做的更改会在函数之外产生影响。
  - 静态成员变量的空间，是类加载时分配到所谓的”静态存储区“的。它的生命期与所在的类相同。

3.3.4 函数调用是的存储映像

◆以下面函数调用为例，说明调用过程中存储映像的变化。

```
public static void main(String[] args) {
    int i = 5;
    int j = 2;
    int k = max(i, j);
    System.out.println(
    "The maximum between " + i +
    " and " + j + " is " + k);
}
public static int max(int num1, int num2) {
    int result;
    if (num1 > num2)
    result = num1;
    else
    result = num2;
    return result;
 }
```

3.3.4 函数调用是的存储映像

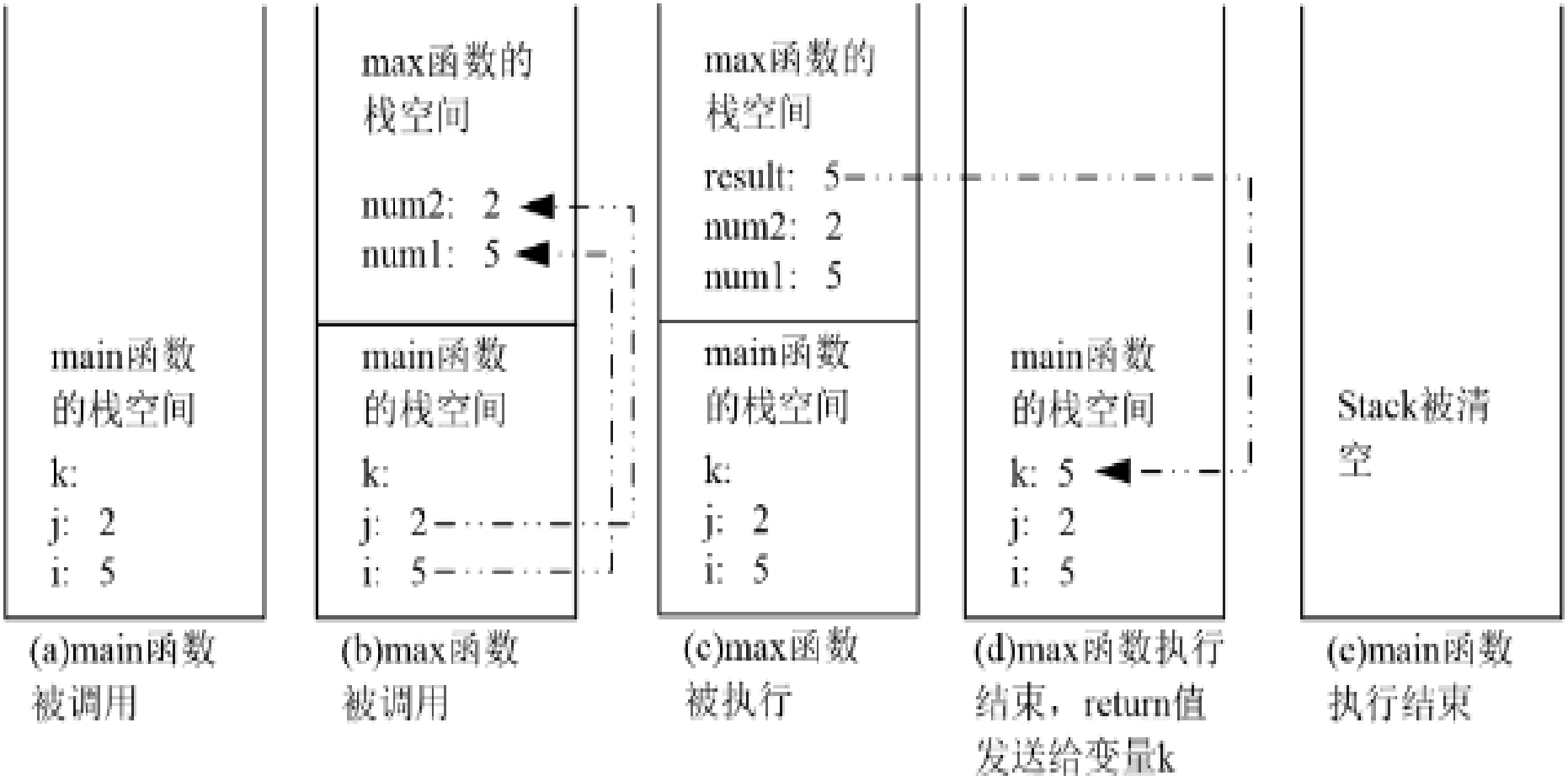


图 3.2 函数调用过程中栈的变化

3.3.4 函数调用是的存储映像

- (a) JVM加载类，并调用main()。调用main时首先为main分配栈空间(stack frame), 并在栈中i, j, k分配位置。然后i, j的值设置为2和5.
  - 因为main()是static的，可以直接调用。
- (b) int k = max(i, j)执行时，即函数调用发生，JVM 为 max 分配栈空间，在栈中为形参 num1 和 num2 分配空间，同时将实参 i 和 j 的值 5,2 分配传递给 num1 和 num2。
- (c) max 函数开始执行，max 函数的局部 result 被分配空间。当 max 函数执行到末尾，result被赋值 5。
- (d) max 函数执行到 return，即函数结束，max 栈空间被释放，result 的值“5”被发送给调用函数 main 并赋值给局部变量 k。
- (e) main 函数结束，main的栈空间释放

3.3.5 重载(Overloading)

- 方法名和参数类型一起，构成方法签名（注意不包括返回值类型）
- 签名不包括函数的返回类型。在同一个类中有两个具有相同签名但具有不同返回类型的函数是非法的。
- 比如方法声明为：

```
static void doTask(int N, double x, boolean test)
```

- 则方法签名为：

```
doTask(int, double, boolean)
```

- Java允许同一类中的两个不同函数具有相同的名称，只要它们的签名不同就可以。这就是方法重载。

3.3.5 重载(Overloading)

- 方法名和参数类型一起，构成方法签名（注意不包括返回值类型）
- 签名不包括函数的返回类型。在同一个类中有两个具有相同签名但具有不同返回类型的函数是非法的。
- 比如方法声明为：

```
static void doTask(int N, double x, boolean test)
```

- 则方法签名为：

```
doTask(int, double, boolean)
```

- Java允许同一类中的两个不同函数具有相同的名称，只要它们的签名不同就可以。这就是方法重载。

3.3.5 重载(Overloading)

◆System.out 就包括重载的特性，该对象包括许多名为 println 的不同方法，这些方法都有不同的签名。

println(int) println(double)

println(char) println(boolean)

println() 等等

◆方法重载又叫编译时多态, 通过方法重载实现了”静态多态性“。

- ■同一个名字，不同的动作。
- ■编译器根据调用参数的类型，确定应该调用同名方法的哪个版本。

3.3.6 函数例子

◆例 1：编写一个函数来计算和输出给定正整数的所有除数，函数的参数是正整数。

■假设 N 的值确定，那么算法是：“对于从 1 到 N 范围内的每个可能的除数 D，如果 D整除 N，则 D 为 N 的一个除数，输出 D。

```
static void printDivisors( int N ) {
    int D; // N 的一个可能的除数.
    System.out.println("The divisors of " + N + "
are:");
    for ( D = 1; D <= N; D++ ) {
        if ( N % D == 0 ) // N 是否可以被 D 整除?
        System.out.println(D);
    }
}
```

3.3.6 函数例子

◆例 2：编写一个名为 printRow 的私有函数，它有一个 char 类型的参数 ch 和一个 int 类型的参数 N，该函数输出一行包含 N 个字符 ch 的文本

```
private static void printRow( char ch, int N ) {
    int i; // 循环的控制变量.
    for ( i = 1; i <= N; i++ ) {
        System.out.print( ch );
    }
    System.out.println();
}
```

3.3.6 函数例子

◆例 3：这是一个以字符串为参数的例子，对于字符串中的每个字符，它应该输出一行包含该字符的 25 个重复输出，程序要求使用 printRow()函数来产生输出。

■首先确定函数名称并为参数选择一个名称，即：printRowsFromString 和参数 str。该算法逻辑是：对于字符串 str 中的每个位置 i，调用 printRow(str.charAt(i),25)以输出一行字符

```
private static void printRowsFromString( String
str ) {
    int i; // 循环控制变量.
    for ( i = 0; i < str.length(); i++ ) {
        printRow( str.charAt(i), 25 );
    }
}
```

3.3.6 函数例子

◆调用的main方法如下。main()、printRowsFromString()和printRow()这三个函数必须集中在同一个类中。

```
public static void main(String[] args) {
    Scanner scanin = new Scanner(System.in);
    String inputLine; // Line of text input by user.
    System.out.print("Enter a line of text: ");
    inputLine = scanin.nextLine();
    System.out.println();
    printRowsFromString( inputLine );

    scanin.close();
}
```

3.3.7 数组参数

- 函数的参数类型可以是数组类型，即可以将数组作为单个参数传递给函数。
- 数组是引用类型，所以在栈中传递的是数组的引用。
- 函数内部修改了参数数组的元素，实参数组的对应元素也就被修改了。
- 函数内部可以使用数组的length属性获得数组长度。

3.3.7 数组参数


```
public class E0305ArrayParameter {
    public static void main(String[] args) {
        int x = 1; // x 代表一个整数值
        int[] y = new int[10]; // y 代表一个整数数组
        m(x, y); // 使用参数 x 和 y 来调用函数 m
        System.out.println("x is " + x);
        System.out.println("y[9] is " + y[9]);
    }
    public static void m(int number, int[] numbers) {
        number = 1001; // 给形参 number 赋值
        numbers[numbers.length-1] = 5555; // 给形参 numbers[0]赋值
    }
}
```

```
$java E0305ArrayParameter.java
x is 1
y[9] is 5555
```

3.3.8 命令行参数

- 当使用命令行运行 Java 程序时，用户输入命令来告诉系统执行程序，此时可以在命令中包含程序名以外的其它输入，这些输入称为命令行参数。Java 解释器接受命令行参数后，将它们放入 main 函数中 String[ ]类型的字符串数组中。

3.3.8 命令行参数

```
public class E0306CLIDemo {
  public static void main(String[] args) {
    System.out.println("You entered " + args.length+ "
command-line arguments");
    if (args.length > 0) {
        System.out.println("They were:");
        int i;
        for ( i = 0; i < args.length; i++ )
        System.out.println(" " + args[i]);
    }
  } // end main()
} // end class CLDemo
```

```
$java E0306CLIDemo.java one tow three
You entered 3 command-line arguments
They were:
 one
 tow
 three

$java E0306CLIDemo.java 1 2 "Hello World" "at 4"
You entered 4 command-line arguments
They were:
 1
 2
 Hello World
 at 4
```

3.3.9 抛出异常

- 函数定义说明函数的“要求”，函数的调用者按照要求输入参数，函数成功执行，但是，当调用者没有按照要求提供参数，函数就会抛出异常。
  - 例如：内置函数 Double.parseDouble 的要求参数应该是双精度类型数字的字符串表示；如果调用者通过传递无效字符串作为实际参数，则函数通过抛出 NumberFormatException 类型的异常来响应。
- 针对自己定义的函数，也希望函数在执行中出现错误时通过 throw 语句来抛出异常。
  - throw new IllegalArgumentException(<error-message>);
  - 以上语句产生一个 IllegalArgumentException 异常对象，<error-message>是描述已检测到的错误的字符串
  - 函数中将检查参数的值是否合法。如果不合法，抛出异常。

3.3.9 抛出异常

◆例如，考虑章前面的 print3NSequence 函数。print3NSequence 的参数应该是一个正整数，当实参不是正数的情况下，抛出异常。修改函数定义，使其在违反此条件时抛出异常：

```
static void print3NSequence(int startingValue) {
  if (startingValue <= 0) // 参数不正确
    throw new IllegalArgumentException( "Starting value must be positive." );
    ... //其余代码与前面内容一样，这里省略。
```

◆这是面向对象程序的通常做法。与此对照，C语言函数是通过返回错误代码的形式处理参数错误的。

- EINVAL (Invalid argument)

3.4 返回值

◆函数的返回值是通过 return 语句完成的，语法如下：

return [<expression>]

◆<expression>的类型必须与为函数指定的返回类型匹配。更准确地说，它必须是一个可以合法地分配给一个变量的表达式，该变量的类型由函数的返回类型指定。

◆当返回值类型为void时，直接用return即可。

◆功能：计算表达式，终止函数的执行，回到程序中调用函数的位置。如果有返回值，则使用返回值代替调用位置的函数调用。

◆return可以出现在函数中任何位置。

3.4 返回值

◆例如，对于如下函数：

```
static double pythagoras(double x, double y) {
    // 计算右斜边的长度
    // 三角形的边为 x 和 y.
    return Math.sqrt( x*x + y*y );
}
```

◆语 句 “ totalLength = 17 + pythagoras(12,5); ” ：

◆ 当 到 达 函 数 调 用pythagoras(12,5)时，它将实际参数 12 和 5 分配给函数中的形式参数 x 和 y。

◆在函数体中，它计算 Math.sqrt(12.0*12.0 + 5.0*5.0)，结果为 13.0。

◆该值由函数“返回”，因此 13.0 实质上替换了赋值语句中的函数调用，与语句“totalLength = 17+13.0”具有相同的效果。

◆最终，返回值与 17 相加，即结果 30.0 存储在变量 totalLength 中。

3.4.2 return举例

◆(1)返回int

■下面是一个非常简单的函数，可用于计算 3N+1 个序列。在上一节中，已经讲述了 3N+1序列问题。给定 3N+1 序列中的一项，以下函数计算序列的下一项. 如果喜欢单一return,可以用第二种方法：

```
static int nextN(int currentN) {
    if (currentN % 2 == 1) // 判断 N 是否为奇数
        return 3*currentN + 1; // 计算奇数的序列并返回
    else
        return currentN / 2; // 计算偶数序列并返回
}
```

```
static int nextN(int currentN) {
    int answer; // 返回值
    if (currentN % 2 == 1) // 判断 N 是否为奇数
        answer = 3*currentN+1; // 计算奇数的序列
    else
        answer = currentN / 2; //计算偶数序列
    return answer; // 返回 answer!
}
```

3.4.2 return举例

◆(2) 返回char:

■下面的例子是五等级分割，计算与给定数字等级相对应的等级:

```
static char letterGrade(int numGrade) {
    if (numGrade >= 90)
    return ’A’; // 成绩高于 90 转换为 A
    else if (numGrade >= 80)
    return ’B’; // 80 到 89 之间转换为 B
    else if (numGrade >= 65)
    return ’C’; // 65 到 79 之间转换为 C
    else if (numGrade >= 50)
    return ’D’; // 50 到 64 之间转换为 D
    else
    return ’F’; //其余转换为 F
} // end of function letterGrade
```

3.4.2 return举例

◆(3) 返回boolean：下面的例子判断一个输入数是否是质数。

```
static boolean isPrime(int N) {
    int divisor; // 测试一个数，看看它是否能将 N 整除.
    if (N <= 1)
        return false; // N<= 1 不是质数.
    int maxToTry; // 测试的最大除数
    maxToTry = (int)Math.sqrt(N);
    // 从 2 到 maxToTry，逐一检查它是否整除 N，
    //如果无法整除 N，那么 N 是质数
    for (divisor = 2; divisor <= maxToTry; divisor++) {
        if ( N % divisor == 0 ) // 检查 divisor 是否将 N 整除.
            return false; // IN 不是质数
    }
    //程序执行到这里，说明 2 到 maxToTry 的每个数都不能整除 N.
    //N 是质数
    return true; // Yes, N is prime.
} // end of function isPrime
```

3.4.2 return举例

◆(4) 返回String：

■函数输入一个字符串作为参数，返回值是参数的反向副本

```
static String reverse(String str) {
    String copy; // 保存反转字符.
    int i; // 记录字符串 str 中的位置,从 str.length() - 1 到
0.
    copy = ""; // 初始化为””字符.
    for ( i = str.length() - 1; i >= 0; i-- ) {
        // 将 str 的第 i-th 个字符赋值给 copy.
        copy = copy + str.charAt(i);
    }
    return copy;
}
```

◆3.5.1 工具箱

◆对于软件开发来说工具箱会提供接口，说明工具箱中都有哪些可以利用的工具。这也就是API-Application Programming Interface。

◆Java 提供一组更加复杂的工具箱，工具箱里装满了已经为执行某些任务而编写的函数。这些函数为进行某种复杂计算（例如求解“微分方程”）提供一套 API，以允许其他人使用这些函数，而无需了解他们执行的计算的细节。

- 例如，数学函数 Math.sqrt()、String 数据类型及其相关函数以及 System.out.print()函数等。
- 标准 Java API 也包括用于处理图形用户界面、网络通信、读取和写入文件等的函数。

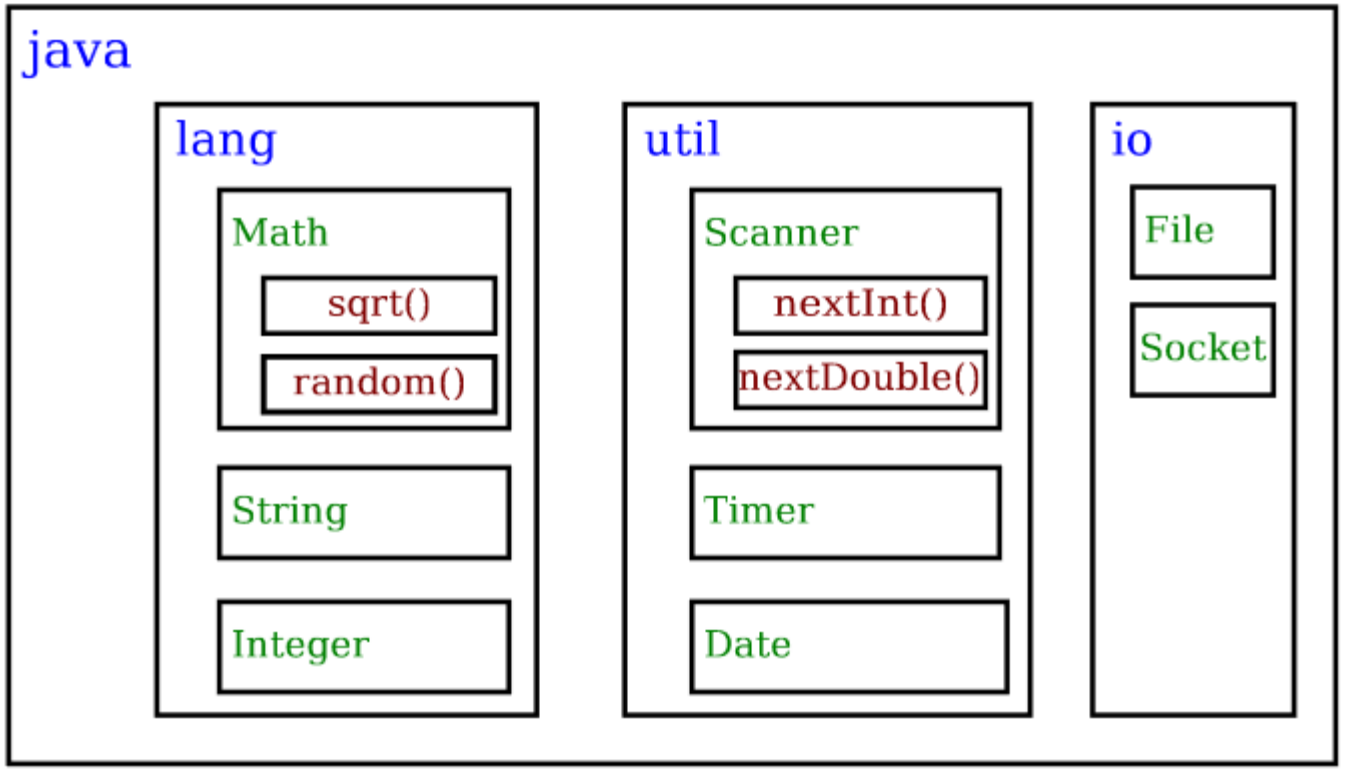
◆Java工具箱包含在各个Java包中。

3.5.2 Java 的标准包

- Java 中的标准 API 中的函数被分组到不同的类中。为了提供更大规模的管理方式，Java中的类可以分组到包中。一个包可以包含类和其他包，包含在另一个包中的包有时称为“子包”。
- 例如，Java 的子包之一被命名为“util”。由于 util 包含在 Java 中，所以它的全名实际上是 java.util。这个包中包含各种实用程序类。
- Java 包中还有其他几个子包，例如提供输入/输出功能的 java.io 和处理网络通信的 java.net。最基本的包称为 java.lang，它包含基本类，例如 String、Math、Integer 和 Double 等。

3.5.2 Java 的标准包

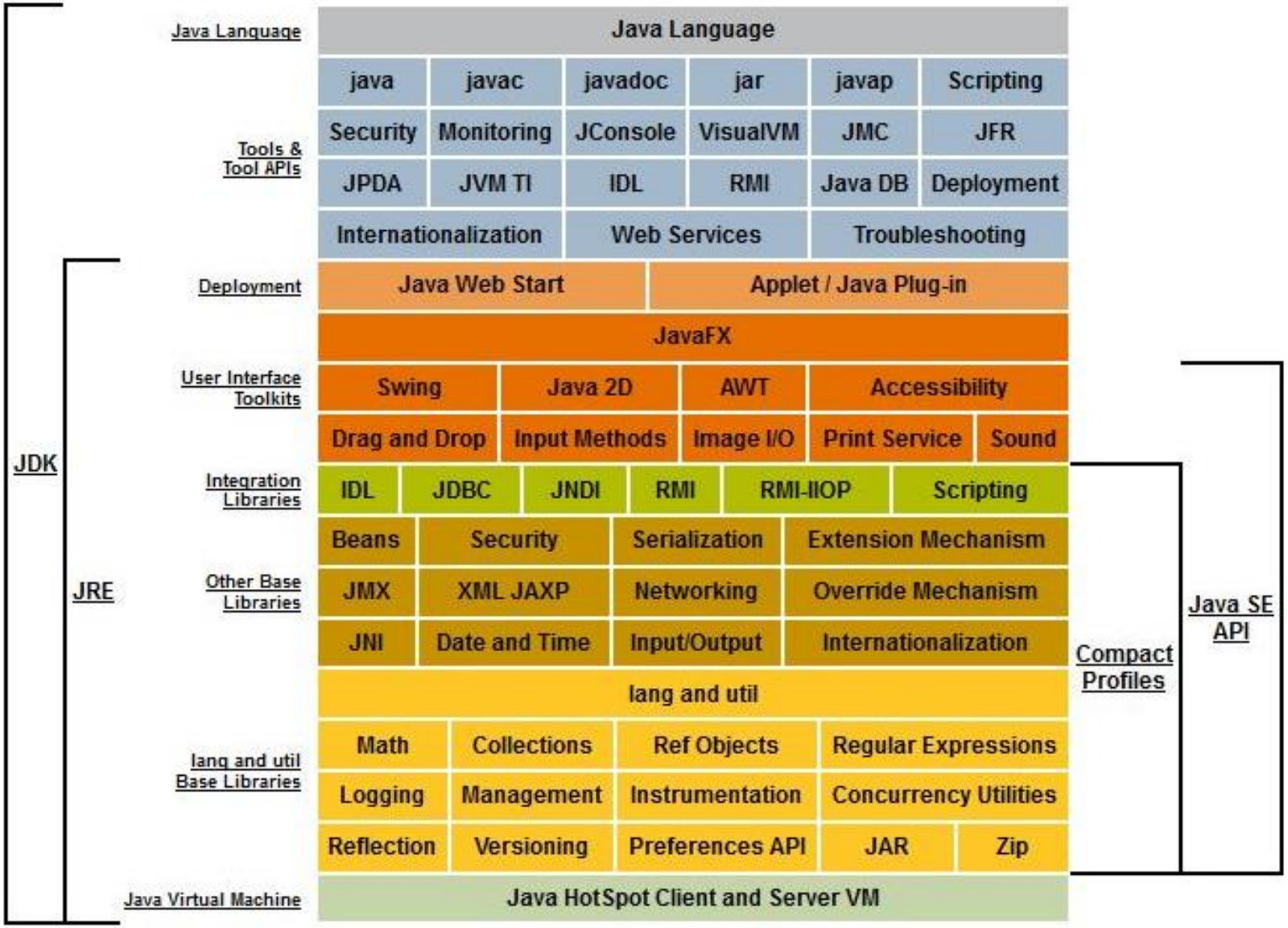
◆Java 包的关系(这只是小部分Java标准包)



Subroutines nested in classes nested in two layers of packages.
The full name of sqrt() is java.lang.Math.sqrt().

3.5.2 Java 的标准包

- https://www.oracle.com/java/technologies/platform-glance.html



3.5.3 使用包中的类

- 使用其他包中的类，就要在代码中写全称，比如如下代码定义了一个变量input, 类型为java.util包中Scanner 类：

```
java.util.Scanner input;
```

- 上面这样写代码太繁琐，可以导入Scanner类之后就不用写Scanner类的全称了。
- import java.util.*;
  - 在文件头部，package语句之后导入：

```
import java.util.Scanner;
```

  - 这样在声明变量时就可以这样写：

```
Scanner input;
```

3.5.3 使用包中的类

◆如果一个包中包含很多类，而且程序亦使用了许多类，可以通过一个通用符号“*”来一次导入多个类，例如，导入 java.util 中包含的所有类，代码如下：

```
import java.util.*;
```

◆当程序开始以这种方式导入大量包时，需要避免类名冲突的发生，即不同包中的两个类可能具有相同的名称。

- 例如，java.awt 包和 java.util 包都包含一个名为 List 的类，如果同时导入java.awt.*和 java.util.*，则定义的变量类型为 List 时，编译器就会输出一条关于类名不明确的编译器错误消息。
- 解决以上类名冲突的方法有两个，一是使用类的全名，java.awt.List 或 java.util.List。另一种解决方法是使用 import 来导入需要的各个类，而不是导入整个包。

3.5.3 使用包中的类

- 因为 java.lang 包是一个基础包，所以 java.lang 中的所有类都会自动导入到每个程序中。
  - 这就是为什么能够使用类名 String 而不是 java.lang.String，以及 Math.sqrt()而不是 java.lang.Math.sqrt()。
- 程序员亦可以创建新的包，如果想把正在编写的某些类归纳于一个指定的包中，那么定义这些类的源代码文件必须以以下格式的行开头：

  package utilities;
- 源代码文件将放置在与包同名的文件夹中，在本示例中为“utilities”。如果有子包，子包中的类必须在子文件夹中。
- 例如，名为 utilities.net 的包中的一个类将位于名为“utilities”的文件夹内的名为“net”的文件夹中。
- 包中的类自动访问同一包中的其他类；也就是说，一个类不必从定义它的包中导入类。
  - "包访问性"（package-private）或者"包级私有"（package-level access）。

3.5.5 Javadoc

- 大多数 Java API 的文档是使用称为Javadoc 的系统来生成的。
- Javadoc 文档是根据 Java 源代码文件中的特殊注释来产生的。
  - /** */
- javadoc 的工具可以读取 Java 源代码文件，提取它找到的任何 Javadoc 注释，并创建一组包含格式良好、相互链接的注释的 Web 页面。
- Javadoc格式
  - Javadoc 注释旨在显示在网页上，所以注释
  - 可以包含 HTML 标记命令。javadoc 工具会将注释说明中的任何 HTML 命令复制到它创建的网页中。
  - 还可以包括 doc 标记，这些标记由 javadoc 工具作为命令处理。文档标签的名称以字符“@”开头。

3.5.5 Javadoc

- 大多数 Java API 的文档是使用称为Javadoc 的系统来生成的。
- Javadoc 文档是根据 Java 源代码文件中的特殊注释来产生的。
  - /** */
- javadoc 的工具可以读取 Java 源代码文件，提取它找到的任何 Javadoc 注释，并创建一组包含格式良好、相互链接的注释的 Web 页面。
- Javadoc格式
  - Javadoc 注释旨在显示在网页上，所以注释
  - 可以包含 HTML 标记命令。javadoc 工具会将注释说明中的任何 HTML 命令复制到它创建的网页中。
  - 还可以包括 doc 标签，这些标签由 javadoc 工具作为命令处理。文档标签的名称以字符“@”开头。

3.5.5 Javadoc

◆Javadoc标签：常用的是红色的四个。

- @param：后面跟着一个参数名和描述，用于描述方法参数。
- @return：后面跟着描述，用于描述方法返回值。
- @throws 或 @exception：后面跟着异常类名和描述，用于描述方法可能抛出的异常。
- @see：提供指向其他主题的链接，比如另一个类、方法或者网页。
- @since：表明从哪个版本开始引入了类、方法、字段等。
- @deprecated：表明某个元素已不再推荐使用，可能在将来的版本中删除。
- @version：描述类或接口的版本。
- @author：类或接口的作者。
- @code：用来表示代码片段的内联标签，通常用在描述中。
- @link：用于内联链接到其他Javadoc文档页面的元素。
- @linkplain：与@link类似，但链接的文本是“plain”，不显示任何代码样式。
- @summary：用于为注释内容提供一个简短的总结。

3.5.5 Javadoc

- 集成开发环境基本都支持Javadoc. 也可以在命令行单独使用。
- 即便不打算生成文档，自己写的代码也应该尽量添加Javadoc注释。
- 举例： `E0307SimpleCalculator.java`

3.5.5 Javadoc: 代码后半部分省略

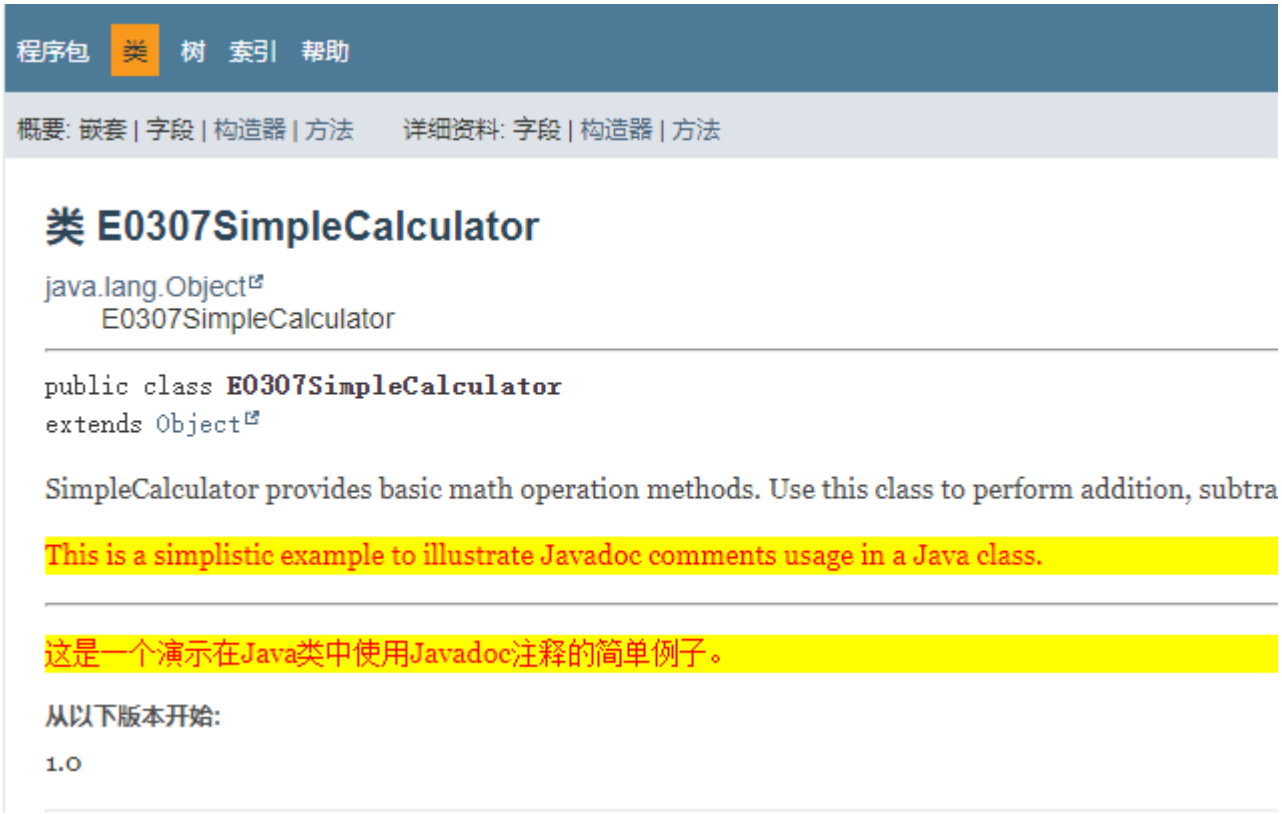

```
/**
 * SimpleCalculator provides basic math operation methods.
 * Use this class to perform addition, subtraction, multiplication, and
division.
 * <p style="color: red; background-color: yellow;">
 * This is a simplistic example to illustrate Javadoc comments usage in a
Java class.
 * </p> <hr>
 * <p style="color: red; background-color: yellow;">
 * 这是一个演示在Java类中使用Javadoc注释的简单例子。
 * </p>
 *
 * @author 陈测试
 * @version 2.0
 * @since 1.0
 */
public class E0307SimpleCalculator {
    /**
     * Adds two integers.
     *
     * @param number1 The first number to add.
     * @param number2 The second number to add.
     * @return The sum of number1 and number2.
     */
    public int add(int number1, int number2) {
```

3.5.5 Javadoc

◆在命令行生成javadoc:

■javadoc -encoding UTF-8 -d E0307_Javadoc E0307SimpleCalculator.java

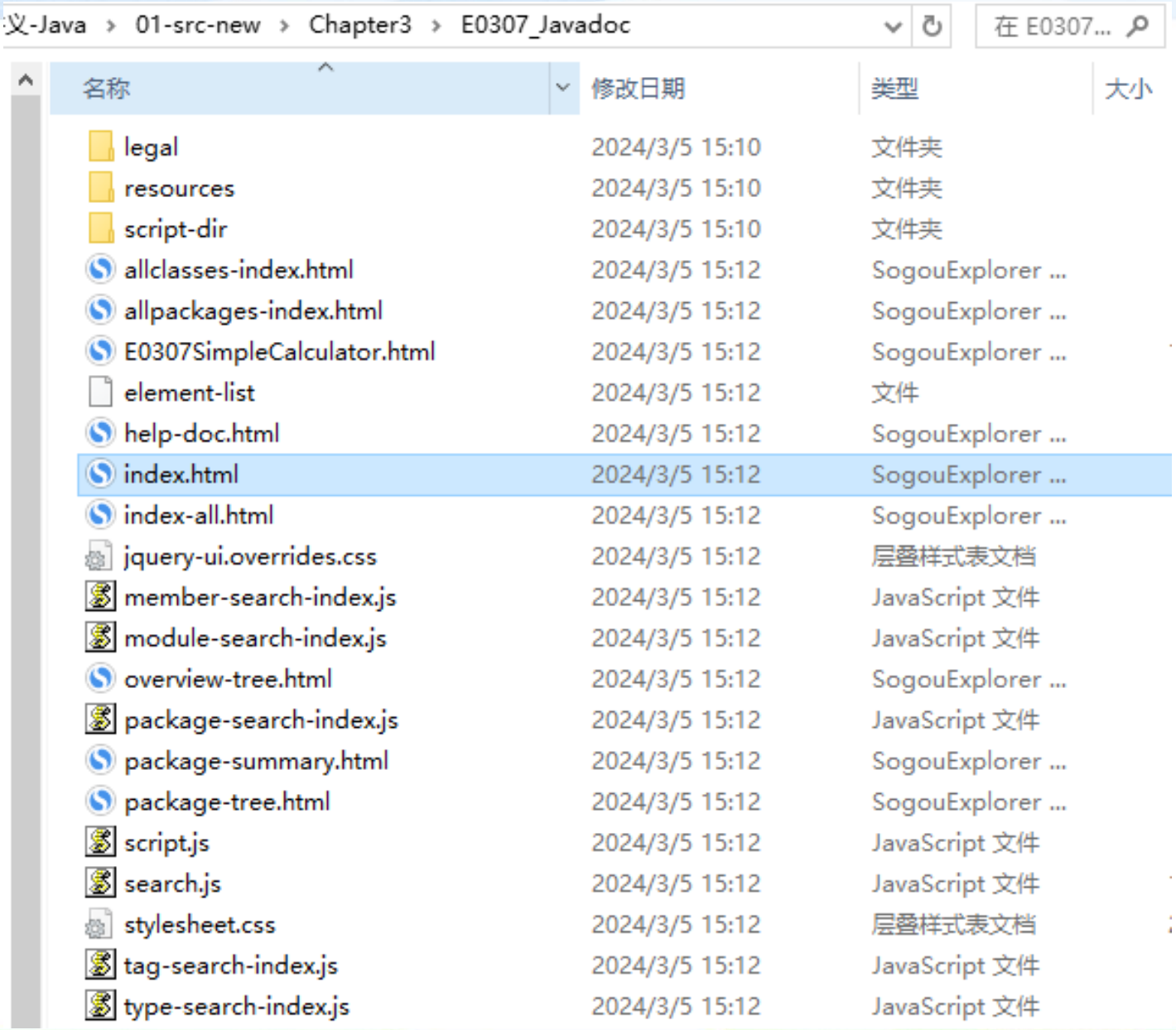
◆生成的文档打开后(index.html):



3.5.5 Javadoc

◆生成的文件：

（入口：index.html）



3.5.6 静态导入*

- Import指令可以使得类名不必写全称。但是静态成员变量和静态成员方法还是需要类名修饰，比如：
  - System.out (静态成员变量)
  - Math.sqrt （静态方法）
- 为了解决这个问题，使用静态导入：
  - `import static` <package-name>.<class-name>.<static-member-name>; （从类中导入一个静态成员名称）
  - `import static` <package-name>.<class-name>.*; （从类中导入所有公共静态成员。）
- 静态导入必须写<package-name>,即使是java.lang包也要写。
  - 结果就说如果类在默认包中，就无法进行静态导入。因为没有包名。

3.5.6 静态导入*

◆比如:

- `import static java.lang.System.out;`
- 然后代码中就可以直接写 out.println, 而不用写System.out.println

◆再比如:

- `import static java.lang.Math.*;`
- 代码中就可以直接用sqrt, log, cos, PI, 而不必Math前缀。

3.6 函数的前置条件和后置条件*

- 锲约式设计是由 Bertrand Meyer 提出的，他是 Eiffel 编程语言的创立者。
- 在编程和软件工程中，函数的前置条件和后置条件是契约式设计（Design by Contract, DbC）中的两个核心概念。契约式设计是一种为确保软件模块（如函数、方法或类）之间正确交互的方法论。
- 函数的前置条件和后置条件通常在文档中描述，这样可以让其他程序员了解怎样使用函数，以及可以期待什么样的结果。

3.6 函数的前置条件和后置条件*

◆前置条件（Preconditions）

- 前置条件是调用函数之前必须满足的条件。它们定义了函数执行所需的环境或状态。
- 如果在调用函数前不满足前置条件，则函数的行为是不可预知的。函数可能无法正常工作，可能会抛出异常或产生不正确的结果。
- 前置条件是函数调用者必须遵守的规则。调用者负责确保这些条件得到满足。
- 例如，一个排序函数可能要求输入的数组必须非空，作为其前置条件。

◆前置条件可能包括但不限于以下情况：

- 参数值必须在特定的范围内。
- 某些资源，如文件或网络连接，必须是可用的。
- 系统或者对象应处于规定的状态

3.6 函数的前置条件和后置条件*

◆后置条件（Postconditions）

- 后置条件是函数执行后应当保证为真的条件。它们定义了函数执行成功后应满足的期望结果。
- 后置条件是函数提供者需要保证的承诺。如果前置条件均已满足，那么执行完毕后后置条件也应当得到满足。
- 例如，一个计算平方根的函数可能保证，对于任何非负的输入值，其返回值的平方等于输入值。

◆后置条件可能包括但不限于以下情况：

- 函数返回特定的结果。
- 某些状态改变已经发生，如数据结构已经更新。
- 某些资源已释放，或者新的资源已经建立。